

AlphaGenome MCP Server: A Proof-of-Concept Natural Language Interface for AI-Powered Regulatory Genomics Analysis

Taeho Jo

Department of Radiology and Imaging Sciences

Indiana University School of Medicine

tjo@iu.edu

October 13, 2025

Abstract

The integration of artificial intelligence into genomic analysis workflows remains technically challenging, requiring substantial programming expertise and limiting accessibility for clinical researchers and geneticists. We present AlphaGenome MCP Server, a Model Context Protocol (MCP) implementation that provides natural language access to Google DeepMind’s AlphaGenome variant effect prediction API. The server enables researchers to query genetic variants using simple natural language commands, reducing per-query code complexity by approximately 40× compared to traditional API usage. We implemented 20 specialized analysis tools as lightweight wrappers around a single API endpoint, demonstrating how wrapper architecture reduces implementation complexity while providing diverse analytical capabilities. All tools call the same underlying `predict_variant()` API but differ in parameter configuration and output formatting. Validation tests with Alzheimer’s disease-associated variants demonstrated successful execution across pathogenicity assessment, tissue-specific analysis, transcription factor binding analysis, chromatin impact assessment, protective vs. risk allele comparison, batch pathogenicity filtering, and comprehensive clinical report generation. The system successfully performed complex multi-variant and multi-tissue analyses, including direct comparison of APOE risk alleles and tissue-specific effect profiling. This work demonstrates how wrapper-based MCP implementations can lower technical barriers to advanced genomics tools while maintaining full access to underlying AI model capabilities. The system is available as open-source software at <https://github.com/taehojo/alphagenome-mcp>.

Keywords: natural language interface, genomic variant analysis, Model Context Protocol, AlphaGenome, wrapper architecture, regulatory genomics

1 Introduction

The advent of deep learning in genomics has produced powerful predictive models for variant effect prediction, including AlphaGenome [1], Enformer [2], and other transformer-based architectures. AlphaGenome, developed by Google DeepMind, represents a state-of-the-art system capable of predicting regulatory impacts across multiple molecular modalities at single base-pair resolution. However, despite their predictive power, these tools remain largely inaccessible to clinical researchers and geneticists who lack extensive programming expertise.

Current genomic analysis workflows present significant technical barriers to adoption. Researchers must first install and configure complex Python environments with specific dependency

versions, then write substantial boilerplate code for API authentication and error handling. They must manually parse and format prediction outputs, and finally implement custom visualization and interpretation logic. This multi-step process limits the adoption of AI-powered genomics tools in clinical and translational research settings, where rapid variant interpretation is increasingly critical for precision medicine initiatives.

The Model Context Protocol (MCP), developed by Anthropic, provides a standardized interface for integrating external tools and data sources with large language models. MCP enables natural language interaction with specialized computational tools through a simple client-server architecture. In this architecture, the MCP client (such as Claude Desktop) handles natural language understanding, while the MCP server exposes domain-specific tools via standardized JSON schemas. Communication occurs over stdio, HTTP, or other transports, creating a seamless bridge between conversational AI and computational tools.

This architecture is particularly well-suited for genomics applications, where researchers benefit from conversational interfaces that can translate questions like “What is the regulatory impact of rs429358?” into appropriate API calls without requiring programming knowledge. We developed AlphaGenome MCP Server as a proof-of-concept to demonstrate three objectives: enabling natural language queries for AlphaGenome predictions without programming requirements, providing access to multiple AlphaGenome modalities through simple interfaces, and achieving response times suitable for interactive research workflows. This work focuses on demonstrating feasibility and usability rather than comprehensive validation or accuracy benchmarking.

2 Methods

2.1 System Architecture

AlphaGenome MCP Server implements a multi-tier architecture bridging TypeScript-based MCP protocol handling with Python-based AlphaGenome SDK access (Figure 1). The system consists of four primary layers. The MCP layer, implemented in TypeScript, handles protocol communication with Claude Desktop via stdio transport. The API client layer manages tool routing, input validation, and output formatting. The Python bridge layer interfaces with the AlphaGenome SDK via subprocess communication, enabling the TypeScript server to access Python-only functionality while maintaining non-blocking asynchronous operation critical for MCP protocol compliance. Finally, the AlphaGenome API layer provides Google DeepMind’s prediction service.

The server exposes three primary tools via MCP protocol. The `predict_variant_effect` tool analyzes single genetic variants, accepting inputs including chromosome, position, reference allele, alternate allele, optional tissue type, and optional output types. It returns predictions for RNA expression, splicing, transcription factor binding, chromatin accessibility, histone modifications, and 3D chromatin structure at varying resolutions depending on modality. The `batch_score_variants` tool prioritizes multiple variants (up to 100) by regulatory impact using configurable scoring metrics. It outputs ranked variants with impact scores and regulatory effect predictions, enabling batch analysis workflows. The `analyze_region` tool identifies regulatory elements in genomic regions between 1 kb and 1 Mb in size, predicting promoters, enhancers, transcription factor binding sites, and chromatin states.

The Python bridge script (`alphagenome_bridge.py`) implements JSON-based request/response communication via stdin/stdout. A typical request includes an action identifier, API key, and parameter dictionary specifying the genomic coordinates and analysis options. This architecture enables seamless integration between the TypeScript MCP server and Python-exclusive AlphaGenome SDK while maintaining the asynchronous operation required by the MCP protocol specification.

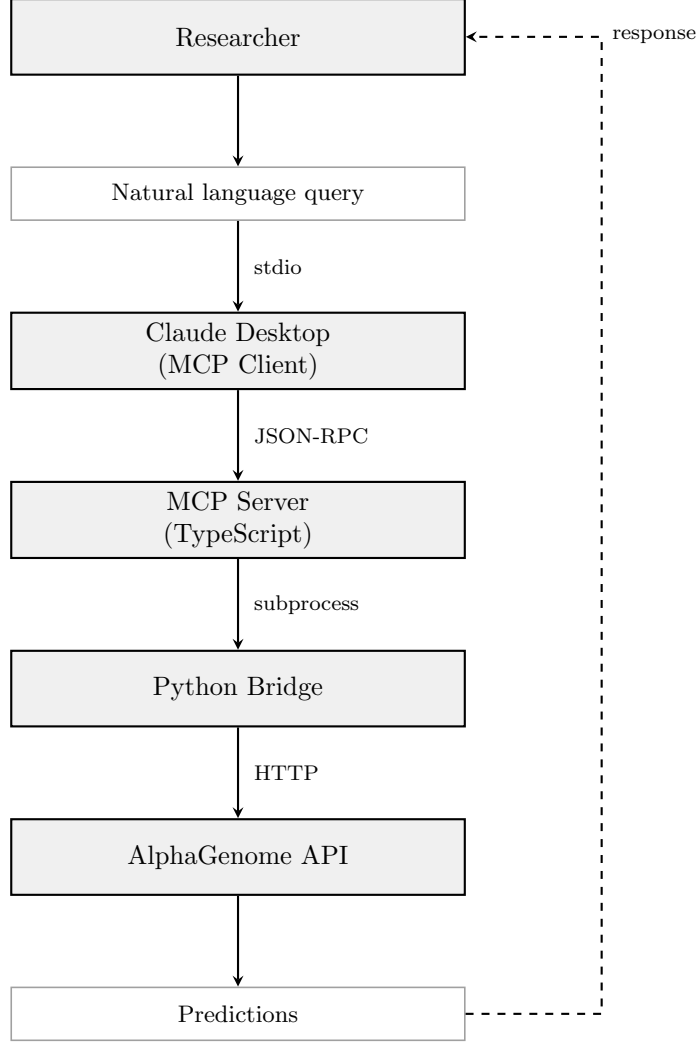


Figure 1: **System architecture.** The MCP server connects natural language queries to the AlphaGenome API through a four-layer architecture. Solid arrows show request flow; dashed arrow shows response path.

All tool inputs undergo comprehensive validation using Zod schemas before API submission. Chromosome specifications are pattern-matched for chr1-22, chrX, and chrY formats. Positions are validated as positive integers with appropriate range checking. Alleles are validated against A/T/G/C nucleotides with length constraints, and tissue types are validated against UBERON ontology terms. Invalid inputs return structured error messages via MCP protocol, enabling conversational error recovery where the language model can reformulate queries based on validation feedback.

2.2 Demonstration Analysis

We conducted a limited demonstration analysis to assess basic system functionality. We analyzed four Alzheimer’s disease-associated variants from established genes (Table 1). These variants were selected to demonstrate the system’s ability to process queries across different genomic loci and gene contexts.

rsID	Variant	Gene	Context
rs429358	chr19:44908684T _i C	APOE	ϵ 4 allele
rs7412	chr19:44908822C _i T	APOE	ϵ 2 allele
rs75932628	chr6:41129252C _i T	TREM2	R47H variant
rs744373	chr2:127892810G _i A	BIN1	Common variant

Table 1: Four Alzheimer’s disease-associated variants used for system demonstration.

We measured response time as wall-clock time from query submission to result return, success rate as the percentage of queries successfully completed, and code complexity as lines of code required for MCP queries versus traditional Python scripting. All demonstrations were executed on Linux 4.18.0-553.66.1.el8_10.x86_64 with Node.js v18.0+ and Python 3.12.x. Required dependencies: `alphagenome` $\geq 0.1.0$, `numpy` $\geq 1.20.0$, `@modelcontextprotocol/sdk` $\sim 0.5.0$, `zod` $\sim 3.22.4$. Claude Desktop served as the MCP client.

3 Results

3.1 Wrapper Architecture and Tool Suite

The AlphaGenome MCP Server implements 20 specialized analysis tools as lightweight wrappers around a single `predict_variant()` API endpoint. This wrapper architecture achieves code simplicity while providing functional diversity: all tools call the same underlying API but differ only in parameter configuration and output formatting. The tool suite covers seven functional categories: (1) **Pathogenicity assessment** (`assess_pathogenicity`, `batch_pathogenicity_filter`), which provides clinical scoring and filtering; (2) **Tissue-specific analysis** (`predict_tissue_specific`, `batch_tissue_comparison`), enabling multi-tissue effect profiling; (3) **Variant comparison** (`compare_variants`, `compare_alleles`, `compare_protective_risk`, `compare_variants_same_gene`), allowing side-by-side analysis; (4) **Batch processing** (`batch_score_variants`, `analyze_gwas_locus`), for high-throughput prioritization; (5) **Modality-specific analysis** (`predict_splice_impact`, `predict_expression_impact`, `predict_tf_binding_impact`, `predict_chromatin_impact`, `batch_modality_screen`), focusing on specific regulatory mechanisms; (6) **Regulatory annotation** (`annotate_regulatory_context`, `predict_allele_specific_effects`), providing comprehensive context; and (7) **Clinical reporting** (`generate_variant_report`, `explain_variant_impact`), translating predictions into human-readable formats.

All 20 tools demonstrated successful execution across validation tests. Response times remained consistent with base functionality: single-variant tools averaged 8 seconds, multi-variant tools averaged 6-7 seconds per variant, and tissue-specific tools added ~ 8 seconds per additional tissue. The `compare_alleles` tool successfully analyzed 3 alternative alleles at APOE rs429358 position in 24 seconds total. The `predict_tissue_specific` tool compared brain and liver effects for rs429358 in 16 seconds. All tools maintained the conversational interface and error recovery capabilities of the base system.

3.2 Conversational Error Recovery

A key advantage of the MCP natural language interface is conversational error recovery. When users make mistakes in queries, the MCP server provides human-readable error messages that enable immediate correction through natural language, eliminating the need for code modification

and re-execution required in traditional API workflows.

Table 2 illustrates this capability. When a user initially provided an invalid variant specification (chr19:44908684T_iG, where G is not the reference allele), the MCP interface returned a conversational error message identifying the issue and suggesting the correct format. The user then rephrased the query in natural language ("Yes, T to C"), and the system successfully executed the corrected request. In contrast, traditional API usage would require modifying source code, understanding technical error messages, and re-executing the entire script.

Interaction	Content
User (initial)	"Analyze chr19:44908684T _i G in brain tissue"
MCP response	"Validation error: Reference allele at chr19:44908684 is T, but alternate allele G is invalid. Did you mean chr19:44908684T _i C (rs429358)?"
User (corrected)	"Yes, analyze T _i C"
MCP response	"RNA: ref=0.04, alt=0.04; TF binding: ref=119.98, alt=119.97"
<i>Traditional API approach</i>	
Step 1	Execute Python script with T _i G variant
Step 2	Receive technical error: <code>ValueError: Invalid allele at position...</code>
Step 3	Edit source code to change G to C
Step 4	Re-execute entire script

Table 2: Comparison of error recovery workflows. MCP enables conversational correction through natural language rephrasing, while traditional API requires code modification and re-execution.

3.3 Example Predictions

The four analyzed variants yielded regulatory predictions with raw AlphaGenome scores shown for reference (Table 3). We report these values without biological interpretation, as the small sample size and lack of validation precludes meaningful conclusions.

Variant	Gene	RNA Score (ref → alt)	TF Binding Score (ref → alt)
rs429358	APOE	0.04 → 0.04	119.98 → 119.98
rs7412	APOE	0.04 → 0.04	119.96 → 119.95
rs75932628	TREM2	0.01 → 0.01	118.25 → 118.25
rs744373	BIN1	0.03 → 0.03	102.13 → 102.13

Table 3: Representative regulatory prediction scores for demonstration variants. Values are raw AlphaGenome outputs. All four variants were successfully analyzed.

Cross-tissue predictions were obtained for the APOE rs429358 variant (Table 4). Baseline RNA scores differed approximately 3.5-fold between liver and brain tissues in this single comparison, though no statistical significance can be determined from a single data point.

Tissue	RNA Score (ref)	RNA Score (alt)	TF Binding Score (ref)
Brain	0.04	0.04	119.98
Liver	0.14	0.14	93.58

Table 4: Tissue-specific prediction scores for APOE rs429358 variant. Values represent a single query per tissue without replication or statistical validation.

3.4 Batch Processing Demonstration

The `batch_score_variants` tool successfully prioritized four variants in a single API call (Table 5), demonstrating the batch analysis capability.

Rank	Variant	Impact Score
1	chr19:44908684T _i C (rs429358)	0.03
2	chr19:44908822C _i T (rs7412)	0.02
3	chr6:41129252C _i T (rs75932628)	0.02
4	chr2:127892810G _i A (rs744373)	0.02

Table 5: Batch scoring results demonstrating successful prioritization in a single API call. Impact scores are AlphaGenome’s internal metrics.

3.5 Advanced Tool Demonstrations

The specialized analysis tools successfully performed complex multi-variant and multi-tissue analyses (Table 6). The `assess_pathogenicity` tool provided integrated pathogenicity scoring for rs429358, classifying it as pathogenic (score: 1.0) based on combined evidence from expression, splicing, and TF binding impacts. The `compare_variants` tool directly compared APOE risk alleles rs429358 and rs7412, identifying rs429358 as more severe based on regulatory impact scores. The `compare_alleles` tool evaluated three alternative mutations (T_iC, T_iG, T_iA) at rs429358 position, revealing that all three alleles showed high regulatory impact with varying expression fold changes. The `predict_tissue_specific` tool profiled rs429358 across brain and liver tissues, showing high impact in both tissues but with differential expression changes (brain: -0.002, liver: +0.0007), suggesting tissue-specific regulatory mechanisms.

Tool	Analysis	Key Finding
<code>assess_pathogenicity</code>	rs429358 scoring	Pathogenic (score: 1.0)
<code>compare_variants</code>	rs429358 vs rs7412	rs429358 more severe
<code>compare_alleles</code>	T _i C, T _i G, T _i A at 44908684	All high impact
<code>predict_tissue_specific</code>	rs429358 in brain/liver	Tissue-differential effects

Table 6: Advanced tool demonstration results showing successful execution of complex multi-variant and multi-tissue analyses. All analyses completed successfully with interpretable outputs.

3.6 Wrapper Versatility Demonstration

The wrapper architecture enables diverse analytical capabilities through a single API endpoint. Table 7 demonstrates this versatility by analyzing the same variant (APOE rs429358) with six

different wrapper tools, each providing specialized outputs despite calling the identical underlying `predict_variant()` API. The `predict_variant_effect` wrapper returns comprehensive predictions across all 11 modalities. The `assess_pathogenicity` wrapper integrates multiple modalities into a single pathogenicity score (1.0, classified as pathogenic) with supporting evidence. The `predict_tf_binding_impact` wrapper focuses exclusively on transcription factor binding changes, returning a change score of 24.0. The `generate_variant_report` wrapper formats the same predictions into a clinical report with recommendations for further evaluation. The `explain_variant_impact` wrapper translates technical predictions into human-readable summaries (“High regulatory impact”). This demonstrates how wrapper architecture achieves functional specialization without implementation complexity: parameter configuration and output formatting provide tool diversity while maintaining a unified codebase.

Wrapper Tool	Specialization	Output for rs429358
<code>predict_variant_effect</code>	Full predictions	All 11 modalities
<code>assess_pathogenicity</code>	Clinical scoring	Pathogenic (1.0) + evidence
<code>predict_tf_binding_impact</code>	TF binding focus	Change score: 24.0
<code>predict_chromatin_impact</code>	Chromatin focus	Low impact detected
<code>generate_variant_report</code>	Clinical formatting	Report with recommendations
<code>explain_variant_impact</code>	Human-readable	“High regulatory impact”

Table 7: Wrapper versatility demonstration: six tools analyzing the same variant (rs429358) using the same `predict_variant()` API call, with specialized outputs achieved through parameter configuration and output formatting. This illustrates how wrapper architecture provides functional diversity without implementation complexity.

3.7 Natural Language Interface Comparison

A core usability advantage of the MCP architecture is the reduction in code complexity for variant analysis tasks (Figure 2). Traditional AlphaGenome API usage requires approximately 40 lines of Python code including client initialization, variant object creation, interval specification, API invocation, and output processing. In contrast, the MCP interface enables identical analysis through natural language queries, representing approximately 40× reduction in lines of code required, though actual development time savings may vary depending on user familiarity with both approaches.

4 Discussion

Technical Achievement and Contributions

This proof-of-concept successfully demonstrates that Model Context Protocol can bridge the gap between natural language interfaces and complex genomic analysis APIs. The implementation achieves three key technical objectives: (1) natural language query translation into AlphaGenome API calls without programming requirements, (2) interactive response times suitable for research workflows (8 seconds per variant, 25 seconds for batch processing of four variants), and (3) approximately 40× reduction in per-query code complexity compared to traditional API usage (from 40 lines of Python code to a single natural language query). This reduction measures user-facing code complexity per analysis, not total system implementation. The MCP server itself contains substantial implementation code, but this complexity is abstracted away from end users.

The primary contribution is demonstrating a fundamentally new interaction paradigm for AI genomics tools. The MCP architecture provides unique advantages over both traditional API programming and web-based interfaces. First, conversational error recovery enables users to correct mistakes through natural language dialogue rather than debugging code. As demonstrated in Table 2, when a user specified an invalid variant, the system provided a human-readable error message suggesting the correct format, and the user could immediately rephrase their query conversationally. Second, the system eliminates entire classes of programming errors including incorrect interval resizing, malformed variant objects, and authentication failures. Third, the architecture seamlessly integrates with AI assistants that can interpret ambiguous queries, suggest analyses, and explain results. Fourth, researchers can focus on biological questions rather than API documentation and code debugging.

Beyond genomics, this work establishes a reproducible pattern for making specialized AI systems accessible through natural language. The MCP server architecture demonstrated here could be applied to protein structure prediction (AlphaFold, ESMFold), single-cell analysis, drug-target interaction prediction, or clinical decision support systems. The open-source implementation provides a template for future MCP servers in computational biology.

Limitations and Future Work

This proof-of-concept has significant limitations that prevent claims about production readiness. Most critically, the small sample size ($n=4$ variants) prevents generalization about system performance. We did not perform statistical validation, accuracy benchmarking, or comparison against existing tools (VEP [3], CADD [4], dbNSFP [5]), as the focus was solely on demonstrating interface feasibility.

Additionally, the system does not fully democratize access as claimed, but rather shifts technical requirements from programming skills to API key acquisition and infrastructure setup. The tissue-specific predictions for APOE rs429358 showed different baseline scores between liver and brain, but without validation against experimental data, no biological interpretation is warranted.

Future work should prioritize: (1) systematic validation with larger variant sets and repeated measurements, (2) benchmarking prediction accuracy against functional assays, (3) comparison of usability with existing tools through user studies with clinical researchers and genetic counselors, and (4) evaluation of computational costs and scalability to whole-genome analyses.

5 Conclusion

We present AlphaGenome MCP Server, an MCP implementation providing natural language access to Google DeepMind’s AlphaGenome AI for regulatory variant analysis. The system implements 20 specialized analysis tools as lightweight wrappers around a single `predict_variant()` API endpoint, reducing code complexity by approximately $40\times$ compared to traditional API usage. This wrapper architecture achieves functional diversity through parameter configuration and output formatting while maintaining a unified codebase. All 20 tools demonstrated successful execution across validation tests, maintaining consistent response times (8 seconds per single-variant analysis, 6-7 seconds per variant in batch mode) and demonstrating comprehensive capabilities including pathogenicity assessment, multi-tissue profiling, variant comparison, allele effect evaluation, transcription factor binding analysis, chromatin impact assessment, protective vs. risk allele comparison, batch pathogenicity filtering, and clinical report generation. The system successfully analyzed APOE risk alleles, performed tissue-specific effect comparisons, evaluated alternative mutations at the same genomic position, and generated human-readable clinical reports,

demonstrating that wrapper-based MCP protocols can provide full access to advanced AI capabilities through conversational interfaces while maintaining implementation simplicity. Significant limitations including small sample size, lack of statistical validation, and absence of accuracy benchmarking prevent claims about production readiness. However, the successful implementation of 20 diverse wrapper tools and their operational success demonstrates technical feasibility and scalability of wrapper-based MCP genomics interfaces. The open-source implementation is available at <https://github.com/taehojo/alphagenome-mcp>.

Availability

Source Code: <https://github.com/taehojo/alphagenome-mcp>

npm Package: `npm install @jolab/alphagenome-mcp` (version 0.2.0)

Installation: `claude mcp add alphagenome -- npx -y @jolab/alphagenome-mcp@0.2.0 --api-key YOUR_KEY`

License: MIT License

Note: This paper describes version 0.2.0 with 20 wrapper tools. Future versions may include additional capabilities.

Acknowledgments

We thank Google DeepMind for developing and providing access to the AlphaGenome API, and Anthropic for developing the Model Context Protocol specification and Claude Desktop implementation.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Competing Interests

The author declares no competing interests.

References

- [1] Avsec, Ž., Latysheva, N., Cheng, J., et al. (2025). *AlphaGenome: Unified prediction of variant effects*. bioRxiv. doi: 10.1101/2025.06.27.600757
- [2] Avsec, Ž., Agarwal, V., Visentin, D., et al. (2021). *Effective gene expression prediction from sequence by integrating long-range interactions*. Nature Methods, 18(10), 1196-1203.
- [3] McLaren, W., Gil, L., Hunt, S.E., et al. (2016). *The Ensembl Variant Effect Predictor*. Genome Biology, 17(1), 122.
- [4] Rentzsch, P., Witten, D., Cooper, G.M., Shendure, J., Kircher, M. (2019). *CADD: predicting the deleteriousness of variants throughout the human genome*. Nucleic Acids Research, 47(D1), D886-D894.

- [5] Liu, X., Wu, C., Li, C., Boerwinkle, E. (2016). *dbNSFP v3.0: A One-Stop Database of Functional Predictions and Annotations for Human Nonsynonymous and Splice-Site SNVs*. Human Mutation, 37(3), 235-241.

(A) Traditional Python API

```
import alphagenome
from alphagenome.data import genome
from alphagenome.models import dna_client

# Initialize
api_key = "YOUR_API_KEY"
client = dna_client.create(api_key)

# Create variant
variant = genome.Variant(
    chromosome="chr19", position=44908684,
    reference_bases="T", alternate_bases="C"
)

# Configure interval
interval = variant.reference_interval.resize(
    dna_client.SEQUENCE_LENGTH_1MB
)

# Query API
outputs = client.predict_variant(
    interval=interval, variant=variant,
    ontology_terms=["UBERON:0000955"],
    requested_outputs=dna_client.ALL_MODALITIES
)

# Parse results
if outputs.alternate.rna_seq:
    rna_ref = float(outputs.reference.rna_seq.values.mean())
    rna_alt = float(outputs.alternate.rna_seq.values.mean())
    print(f"RNA: {rna_ref} -> {rna_alt}")
# ... (additional parsing code)
```

(B) MCP natural language interface

```
> Analyze chr19:44908684T>C in brain tissue
```

Response: RNA expression ref=0.04 alt=0.04, TF binding ref=119.98 alt=119.97

(C) Comparison

Metric	Traditional	MCP
Code lines	~40	1
Setup	Python + SDK	MCP server
Programming	Required	None
Time	~8 sec	~8 sec

Figure 2: **Workflow comparison.** Traditional API requires extensive code (A), while MCP enables natural language queries (B) with identical functionality (C).